A common mistake is to use single quotations ('Hello!') to express a string. This often ends with confusion and frustration because `"hello" != 'hello'`. Single quotation marks are used to represent Erlang's version of a string (charlist) and are really only used in Elixir code that directly interacts with Erlang code.

List

Lists are expressed using brackets—`[1, 2, 3]`. The next chapter has a section dedicated to them.

Tuple

Tuples are fixed-size lists that are expressed using braces—`{1, 2, 3}`. The differences between tuples and lists are covered in the next chapter.

Map

Maps are key-value dictionaries expressed with percent-brace—`%{a: "map"}`. They also appear in the next chapter.

We'll dig deeper into several of these data types throughout the chapter. First, let's go over the operators that are built into Elixir.

Operators

Elixir comes with a set of basic operators that largely work the way that you would expect. Most operators will feel similar to the operators you use in Ruby, and you'll be right at home. But a few of these operators will behave much differently than similar operators in Ruby.

We'll cover the most commonly used Elixir operators, and you'll see the important differences that a few of these operators have compared to Ruby.

Assignment (Match)

`=` is called the match operator. You can use this as you would in Ruby (`my_var = 1`), but it does much more than just assignment.

It's also used for pattern matching and complex structural comparisons (`[a, b] = [1, 2]`). Pattern matching is one of the most important parts of Elixir's syntax, so Chapter 4, Pattern Matching Your Way to Success, on page 49 is dedicated to it.

Comparison

Elixir provides `==`, `!=`, `>`, `>=`, `<`, `<=`, `===`, `!==` operators. These largely work as you would expect, with a couple of twists.